

Trabalho Prático: Processador de Expressões Aritméticas

1. Objetivo

Desenvolver um sistema de processamento de expressões matemáticas que utilize, de forma híbrida, as duas principais implementações de **Pilha**: a **Estática** (baseada em array) e a **Encadeada** (dinâmica). O aluno deverá converter uma expressão da notação infixa para pós-fixa e, em seguida, calcular seu valor numérico.

2. Requisitos de Implementação (Estruturas)

Para este trabalho, você deve implementar obrigatoriamente:

1. **Pilha Estática (Vetor)**: Utilizada na **Fase de Conversão**. O TOPO deve ser, obrigatoriamente, o índice do último elemento inserido (no fim da parte ocupada do vetor), para garantir operações em $O(1)$.
2. **Pilha Encadeada (Lista)**: Utilizada na **Fase de Avaliação**. O TOPO deve ser obrigatoriamente o ponteiro para o primeiro nó da lista, para garantir operações em $O(1)$, sem necessidade de percorrer a estrutura.

3. Estratégia de Leitura (Parsing)

A entrada será uma string em C. Para facilitar o processamento e focar na estrutura de dados, adote a seguinte estratégia:

- A expressão de entrada deve conter espaços entre cada token (ex.: `( 3 + 4.5 ) * 2`).
- Utilize a função `strtok(string, " ")` para isolar cada unidade lógica (número, operador ou parêntese).
- Utilize a função `atof()` para converter os tokens numéricos em valores de ponto flutuante antes de empilhá-los na fase de avaliação.

4. Fase I: Conversão Infixa para Pós-fixa

Estrutura: Pilha Estática de Caracteres (`char`).

O Algoritmo de Shunting-yard:

Percorra os tokens da expressão da esquerda para a direita:

1. **Se o token for um operando (número)**: Envie-o imediatamente para a saída.
2. **Se o token for um parêntese de abertura (()**: Faça um `push` na pilha.
3. **Se o token for um operador (+, -, *, /)**:
 - Compare a precedência do operador atual com o operador no topo da pilha.
 - Enquanto o topo tiver **precedência maior ou igual** à atual, realize `pop` da pilha para a saída.
 - *Nota:* O parêntese `(` tem a menor precedência dentro da pilha e nunca é removido por um operador.
 - Após a limpeza, realize o `push` do operador atual na pilha.
4. **Se o token for um parêntese de fechamento)**: Realize `pop` da pilha para a saída até encontrar o parêntese de abertura `(`. Descarte ambos os parênteses.

5. **Ao fim da string:** Desvazie a pilha para a saída usando `pop` até que `isEmpty` seja verdadeiro.

Tabela de Precedência:

- Precedência 0: `(`.
- Precedência 1: `+`, `-`.
- Precedência 2: `*`, `/`.

5. Fase II: Avaliação da Expressão Pós-fixa

Estrutura: Pilha Encadeada de Números Reais (`float` ou `double`).

O Algoritmo de Avaliação:

Percorra os tokens da expressão pós-fixa gerada na Fase I:

1. **Se o token for um número:** Converta-o (`atof`) e realize um `push` na pilha encadeada.
2. **Se o token for um operador:**
 - Realize um `pop` para obter o **segundo operando** (valor B).
 - Realize um segundo `pop` para obter o **primeiro operando** (valor A).
 - *Crucial:* Respeite a ordem nas operações não comutativas: $A - B$ ou A / B .
 - Calcule o resultado e realize um `push` do valor de volta na pilha.
3. **Resultado Final:** Ao término, o resultado da conta será o único elemento restante no topo da pilha.

6. Exemplo de Rastro Detalhado

Entrada Infixa: `( 10 + 2 ) * 3`

Token	Ação (Fase 1 - Estática)	Pilha (Topo à Direita)	Saída Pós-fixa
<code>(</code>	Push <code>(</code>	<code>[(]</code>	
<code>10</code>	Saída	<code>[(]</code>	<code>10</code>
<code>+</code>	Push <code>+</code> (topo é <code>(</code>)	<code>[(, +]</code>	<code>10</code>
<code>2</code>	Saída	<code>[(, +]</code>	<code>10 2</code>
<code>)</code>	Pop até <code>(</code>	<code>[]</code>	<code>10 2 +</code>

*	Push *	[*]	10 2 +
3	Saída	[*]	10 2 + 3
Fim	Esvazia Pilha	[]	10 2 + 3 *

7. Critérios de Avaliação

Sua implementação será avaliada pela corretude lógica e pelo rigor no gerenciamento de recursos:

- **Gestão de Memória:** Cada `pop` na pilha encadeada deve obrigatoriamente chamar `free()` no nó removido para evitar *memory leaks*.
- **Robustez:** O código deve tratar divisões por zero e alertar sobre o limite de capacidade (`MAX`) na pilha estática.
- **Relatório Técnico:** Inclua uma breve análise comparando o overhead de memória de cada nó da pilha encadeada (dado + ponteiro) em relação à alocação contígua da pilha estática.

Interface via Argumentos de Linha de Comando (CLI)

Para esta atividade de mestrado, o programa **não deve ser interativo** (ou seja, não utilize `scanf` ou `gets` para ler a expressão durante a execução). Em vez disso, a expressão deve ser capturada diretamente da linha de comando no momento da invocação do programa.

1. Funcionamento do `argc` e `argv`

A função principal deve ser declarada como `int main(int argc, char *argv[])`.

- **`argc` (Argument Count):** Um inteiro que indica quantos argumentos foram passados (incluindo o nome do próprio programa).
- **`argv` (Argument Vector):** Um array de ponteiros para strings que contém os argumentos.

2. Algoritmo de Captura e Validação

O programa deve seguir este fluxo lógico inicial:

1. **Verificação de Parâmetros:** O programa deve verificar se `argc` é pelo menos 2. Caso contrário, deve exibir uma mensagem de erro que instrua o uso correto e encerrar o programa.
2. **Acesso à Expressão:** O primeiro argumento útil (a expressão completa) estará em `argv[1]`.
3. **Importância das Aspas:** Oriente que, ao testar no terminal, a expressão deve estar entre aspas duplas. Isso garante que o sistema operacional envie a expressão

inteira como um único "token" para o `argv[1]`, em vez de separar cada caractere em argumentos distintos.

Exemplo de Execução no Terminal:

```
./processador "( 10 + 2 ) * 3"
```

3. Integração com o Parsing (`strtok`)

Uma vez capturada a string em `argv[1]`, ela deve ser utilizada como base para as fases de processamento:

- **Fase I (Conversão):** A string de `argv[1]` é lida para alimentar a **Pilha Estática** de operadores.
- **Fase II (Avaliação):** O resultado da conversão alimenta a **Pilha Encadeada** de valores numéricos.

4. Exemplo de Código Base para o `main`

```
int main(int argc, char *argv[]) {  
    // Validação de entrada CLI  
    if (argc < 2) {  
        fprintf(stderr, "Erro: Expressao nao fornecida.\n");  
        fprintf(stderr, "Uso: %s \"( 1 + 2 ) * 3\"\n", argv[0]);  
        return 1;  
    }  
  
    // argv[1] contém a string completa a ser processada  
    char *expressaoOriginal = argv[1];  
  
    // Início do processamento utilizando as estruturas de Pilha  
    printf("Processando a expressao: %s\n", expressaoOriginal);  
  
    // CHAMADA PARA FASE 1 (PILHA ESTÁTICA)  
    // Imprima a expressão convertida (para conferência)  
    // CHAMADA PARA FASE 2 (PILHA ENCADEADA)[cite: 1]  
  
    return 0;  
}
```